Facultad de Ingeniería

Diseño según Plataforma

Trabajo Práctico N.º

Testing de Integración

Campo	Completar
Nombre y apellido del estudiante	
Legajo	
Fecha de entrega	
Versión de Unity utilizada	

Modalidad: individual.
Dedicación estimada: 8 a 10 horas de trabajo fuera de clase.
Proyecto: nave + asteroides (`Asteroides/asteroide-final`), el mismo que se usó en las clases de Testing de Integración.

1. Objetivos

Al completar este trabajo práctico, el estudiante debe demostrar que puede:

1. Identificar los componentes que participan en un flujo del juego y las **costuras** entre ellos (dónde un componente llama a otro, le pasa un dato o depende de una condición).
2. Diseñar casos de prueba de integración **positivos, negativos y de borde** con la ficha de 8 campos vista en clase, sin repetir los tests que ya existen.
3. Implementar tests de integración en Play Mode con el Unity Test Framework, usando componentes reales del juego.
4. Construir una **fixture** (`[UnitySetUp]` / `[UnityTearDown]`) que deje a cada test independiente del estado que dejaron los demás.
5. Reemplazar esperas fijas por **esperas por condición con tiempo máximo** o por pasos de física, cuando corresponda.
6. Comprobar con ejecuciones reales si los tests son **independientes y reproducibles**, y reconocer lo que esas ejecuciones no alcanzan a demostrar.
7. Demostrar que cada test propio **es capaz de fallar** cuando el defecto que vigila está presente.
8. Analizar los resultados: distinguir si una falla es del **test** o del **sistema**, señalar la interacción involucrada y respaldar cada conclusión con evidencia.

2. Contexto

2.1 El juego

La nave se ubica en la parte inferior de la pantalla y dispara láseres hacia arriba. Desde arriba caen asteroides.

- Si un láser choca un asteroide, el asteroide se destruye y el puntaje sube.
- Si un asteroide choca la nave, termina la partida (Game Over).

- El botón de inicio llama a `NewGame()`, que empieza una partida nueva.

Componentes (`Assets/Scripts/`):

Componente	Responsabilidad general
`Game`	Coordina la partida: puntaje, Game Over, nueva partida y textos de la UI.
`Spawner`	Genera asteroides, manualmente o en forma automática cada cierto intervalo.
`Asteroid`	Cae y reacciona al chocar.
`Laser`	Sube y reacciona al chocar.
`Ship`	Movimiento, disparo, explosión y reparación de la nave.

El prefab `Assets/Resources/Prefabs/Game.prefab` contiene la nave (`Ship`, cuyo modelo se llama `ShipModel`), el `Spawner`, una plantilla inactiva del láser y la UI (`UICanvas`, con sus textos). Los asteroides se crean a partir de cuatro prefabs que están en `Assets/Prefabs/`.

Leer y comprender el código de estos cinco archivos es **parte del trabajo**. Este documento no describe sus detalles internos.

2.2 Situación inicial

El proyecto ya trae dos archivos de tests en `Assets/Tests/`:

Archivo	Tests	Estado conocido
`TestSuite.cs`	7 tests de Play Mode (01 a 07)	**VERIFICADO:** pasan 7/7 cuando se ejecutan solos, en batchmode, con Unity 6000.3.11f1.
`IntegrationHypothesesTests.cs`	4 tests de Play Mode (casos INT-AST-13, 14 y 15)	**VERIFICADO:** 3 de los 4 fallan a propósito, porque documentan defectos del juego que no fueron corregidos.

En clase también se diseñaron casos que **no existen** como test (INT-AST-08 a 12 y 16). Su estado es **PROPUESTO**.

Hay aspectos del proyecto que **nadie verificó todavía** (estado **UNKNOWN**). Algunas actividades de este TP los investigan, así que su resultado no se conoce de antemano y cualquier resultado bien documentado es válido:

- si `TestSuite` sigue pasando 7/7 cuando se ejecuta junto con otros archivos de tests;
- si cada test pasa cuando se ejecuta solo;
- en qué orden ejecuta los tests el Test Runner;
- si fijar una semilla con `Random.InitState` alcanza para que el `Spawner` genere siempre los mismos asteroides.

2.3 Requisitos

- Unity **6000.3.11f1** (si usás otra versión, indicala en el encabezado).
- Unity Test Framework **1.6.0**, ya incluido en el proyecto.
- Ejecución desde **Window ▸ General ▸ Test Runner**, pestaña **PlayMode**, con **Run All** y **Run Selected**.
- No hace falta instalar ningún paquete adicional.

3. Uso de las etiquetas de estado en el informe

Toda afirmación del informe sobre el comportamiento del juego o de los tests debe llevar una de estas etiquetas:

Etiqueta	Cuándo usarla
VERIFICADO	Lo comprobaste ejecutando. Indicá cuál es la evidencia (captura, mensaje, log).
APLICADO	Está implementado en tu código, pero ninguna ejecución demuestra su efecto.
PROPUESTO	Es una idea o una corrección que no implementaste ni ejecutaste.
UNKNOWN	No tenés evidencia suficiente para afirmarlo ni para descartarlo.

Una afirmación sin etiqueta, o con la etiqueta VERIFICADO pero sin evidencia, se considera no demostrada.

4. Consigna general

Vas a extender la suite de tests del proyecto con **tus propios tests de integración**, escritos en un archivo nuevo, sin modificar el juego.

El trabajo sigue este recorrido:

analizar → diseñar → implementar → ejecutar → comprobar que los tests pueden fallar → analizar resultados → concluir

Cada actividad usa lo que produjiste en la anterior. Respetá las restricciones de la sección 10 durante todo el trabajo.

5. Actividades

Actividad 1 · Línea de base y análisis de componentes

Objetivo: conocer el estado inicial de la suite y comprender los flujos del juego antes de diseñar casos.

Consigna:

1. Abrí el proyecto sin modificarlo y ejecutá **Run All** en la pestaña PlayMode del Test Runner. Registrá qué tests pasan y cuáles fallan.
2. Analizá estos tres flujos del juego:
   - **Flujo A:** un láser destruye un asteroide.
   - **Flujo B:** un asteroide choca la nave.
   - **Flujo C:** se inicia una partida nueva después de un Game Over.
3. Para **cada flujo**, completá:
   - **Diagrama:** evento → componente → componente → resultado observable. Indicá el archivo y la línea de cada llamada entre componentes.
   - **Costuras:** la lista de costuras del flujo. Para cada una, indicá qué cruza (un método llamado, un dato o una condición).
   - **Contratos implícitos:** al menos un **contrato implícito** del flujo, si existe (una dependencia que no está escrita en ninguna interfaz, por ejemplo un nombre). Si no encontrás ninguno, justificalo.
   - **Efecto sin verificar:** al menos un **efecto observable** del flujo que **ningún** test existente verifica. Para justificarlo, citá los asserts de los tests que cubren ese flujo.

4. Identificá en el código las fuentes de **estado compartido** que pueden afectar a un test que se ejecuta después de otro. Para cada una, indicá dónde está (archivo y línea) y qué podría quedar «sucio» para el test siguiente.

Entregable: sección 1 del informe con la captura de la línea de base, los tres diagramas, las tablas de costuras y la lista de fuentes de estado compartido.

Resultado esperado: tres flujos documentados con referencias exactas al código, al menos un efecto sin verificar por flujo y una lista de fuentes de estado compartido con su ubicación.

Actividad 2 · Diseño de casos de integración

Objetivo: diseñar casos nuevos que cubran lo que la suite actual no prueba.

Consigna: diseñá **6 casos de prueba** con la ficha de 8 campos de la sección 6.1, respetando las reglas de la sección 6.2.

Entregable: sección 2 del informe con las 6 fichas completas.

Resultado esperado: 6 fichas en las que el resultado esperado es verificable por código y está respaldado por el diseño. Las filas que no tienen ese respaldo deben quedar marcadas como pregunta de diseño (⚠).

Actividad 3 · Implementación con una fixture propia

Objetivo: escribir los tests de integración con una fixture que los aísle.

Consigna:

1. Creá el archivo `Assets/Tests/IntegracionTP_<Apellido>.cs`, con una clase de test nueva, dentro del assembly `Tests` que ya existe.
2. Implementá en ese archivo **al menos 4** de tus casos, elegidos según la sección 6.3.
3. Escribí una fixture con `[UnitySetUp]` y `[UnityTearDown]` que garantice que cada test empieza desde un estado conocido y no deja nada al terminar. Tené en cuenta qué hace falta para que:
   - `Start()` ya se haya ejecutado cuando el test empieza;
   - la limpieza alcance todo lo que el test creó, no solo el prefab;
   - las destrucciones se completen antes del test siguiente.
4. Cumplí las reglas de calidad de la sección 6.4.
5. En el informe, explicá **cada decisión** de tu fixture: qué problema concreto del proyecto resuelve y en qué línea del código está ese problema.

Entregable:

- el archivo `IntegracionTP_<Apellido>.cs`;
- la sección 3 del informe con la explicación de la fixture y una tabla con la correspondencia entre caso y test (ID de ficha → nombre del método).

Resultado esperado: el archivo compila en una copia limpia del proyecto y cada test corresponde a una ficha de la Actividad 2.

Actividad 4 · Ejecución, independencia y reproducibilidad

Objetivo: comprobar con evidencia si tus tests son independientes y reproducibles.

Consigna:

1. Agregá a tu `[UnitySetUp]`, **antes** de instanciar el prefab, un `Debug.Log` que informe cuántos `Asteroid` y cuántos `Laser` hay en la escena en ese momento.
2. Ejecutá y registrá los resultados (pasa / falla) de estas corridas:

Corrida	Qué se ejecuta	Repeticiones
R1	Solo tu clase de test	1
R2	Cada uno de tus tests por separado (**Run Selected**)	1 por test
R3	Toda la suite del proyecto (**Run All**: `TestSuite`, `IntegrationHypothesesTests` y tu clase)	3

3. Con los logs del punto 1, determiná si alguno de tus tests empezó con objetos que dejó otro test. Si pasó, indicá en qué corrida y con qué cantidades. Si no pasó, indicá en qué corridas lo comprobaste.
4. Indicá si `TestSuite` mantuvo 7/7 en R3. Este es uno de los puntos que figuran como UNKNOWN en la sección 2.2: informá lo que observaste, sin generalizar más allá de tus corridas.
5. **Investigación sobre la semilla:**
   - Escribí un test auxiliar, ubicado en tu archivo, que registre con `Debug.Log` la posición X de 3 asteroides generados con `SpawnAsteroid()`.
   - Ejecutalo 2 veces **sin** fijar la semilla y 2 veces fijándola con `Random.InitState` en el SetUp.
   - Concluí qué demostraste y qué queda sin demostrar.
6. Si algún resultado cambió entre repeticiones de R3, tratalo como un posible test frágil: indicá cuál es la causa probable entre las vistas en clase (tiempo, estado compartido, orden, azar o física) y qué evidencia la sostiene.

Entregable: sección 4 del informe con la tabla de resultados de R1, R2 y R3, los logs relevantes (texto o captura), la conclusión sobre el estado compartido y la conclusión sobre la semilla, cada una con su etiqueta de estado.

Resultado esperado: resultados de todas las corridas pedidas y conclusiones que no afirman más de lo que muestran esas corridas. Por ejemplo, que un test pase en todas las corridas no demuestra que pase en cualquier orden posible.

Actividad 5 · Comprobar que cada test puede fallar

Objetivo: demostrar que cada test propio detecta el defecto que dice detectar.

Consigna:

1. Para **cada** test implementado en la Actividad 3:
   1. introducí en el código del juego un cambio **temporal** que reproduzca el defecto que el test vigila (el campo «Bug que detecta» de su ficha);
   2. **antes** de ejecutar, anotá qué test o tests de **todo el proyecto** creés que van a fallar;
   3. ejecutá **Run All** y registrá qué tests fallaron realmente;
   4. **revertí** el cambio y comprobá que la suite vuelve al estado de la línea de base.
2. Si alguno de tus tests **no** falló con el defecto presente, explicá por qué y corregí el test (no el juego) hasta que lo detecte. Si no podés lograrlo, documentalo como limitación del test.
3. Indicá si alguno de los cambios **no fue detectado por ningún test existente** (de `TestSuite` o de `IntegrationHypothesesTests`). Explicá qué significa eso para una suite que estaba en verde.

Entregable: sección 5 del informe con una tabla por test que incluya:

- el cambio temporal (archivo, línea, código original y código modificado);

- los tests que predijiste que iban a fallar;
- los tests que fallaron realmente;
- una captura del test propio en rojo;
- la confirmación de que el cambio fue revertido.

Resultado esperado: cada test propio se vio en rojo al menos una vez por el motivo que declara su ficha, y el juego entregado no tiene modificaciones.

Actividad 6 · Análisis de resultados

Objetivo: interpretar cada falla con evidencia.

Consigna: con el resultado de la **última** corrida R3 (en el estado normal del juego, sin cambios temporales), completá la tabla de la sección 7 para:

- **todos** los tests que fallan, incluidos los de `IntegrationHypothesesTests`;
- **al menos 2** de tus tests que pasan.

Entregable: sección 6 del informe con la tabla de la sección 7.

Resultado esperado: cada falla clasificada como falla del test o del sistema, con la interacción involucrada y evidencia propia que la sostiene.

Actividad 7 · Conclusiones y preguntas finales

Objetivo: reflexionar sobre lo realizado a partir de la evidencia propia.

Consigna: respondé las preguntas de la sección 11. Cada respuesta tiene que citar al menos un resultado concreto de tus actividades (número de actividad y dato).

Entregable: sección 7 del informe.

Resultado esperado: respuestas breves (5 a 10 líneas cada una), apoyadas en tus propios resultados y no en afirmaciones generales.

6. Casos de prueba

6.1 Ficha de 8 campos

Campo	Qué debe contener
ID	`TP-01` a `TP-06`
Objetivo	Qué comportamiento del flujo se verifica, en una oración
Componentes	Qué componentes **reales** participan (al menos 2)
Entrada	Qué evento o dato dispara el flujo
Precondiciones	El estado necesario antes de empezar (incluido el estado de la fixture)
Pasos	Acciones concretas, con valores concretos
Resultado esperado	Valores o estados verificables por código; de dónde sale ese resultado (código, diseño o pregunta abierta)
Bug que detecta	Qué defecto concreto haría fallar este caso
¿Por qué integración?	Qué costura verifica

6.2 Reglas para los 6 casos

1. **Distribución:** al menos **2 positivos**, **2 negativos** y **2 de borde**.
2. **Bordes:** los 2 casos de borde deben corresponder a **dimensiones distintas**, elegidas entre las vistas en clase: valor/posición, tiempo, cantidad/simultaneidad, repetición y orden.

3. **Flujos:** entre los 6 casos tienen que estar cubiertos los **tres flujos** de la Actividad 1.
4. **Originalidad:**
   - Ningún caso puede verificar lo mismo que un test existente (01 a 07, INT-AST-13, 14 y 15).
   - Como máximo **1** caso puede corresponder a uno de los casos diseñados en clase (INT-AST-08 a 12). Si usás uno, indicá cuál.
5. **Caso positivo con varios efectos:** al menos **1** caso positivo debe verificar **2 o más efectos** del mismo flujo sobre componentes distintos.
6. **Asserts que discriminen:** en cada caso, el resultado esperado tiene que ser **distinto del estado inicial**. Si el valor esperado ya era cierto antes del evento, el caso no demuestra que el evento haya ocurrido. Para cada caso, indicá cuál es el estado inicial de lo que vas a comprobar.
7. **Preguntas de diseño:** si el resultado esperado no se puede decidir leyendo el código ni tiene respaldo de diseño, marcá la ficha con ⚠ y escribí la pregunta que habría que hacerle al diseño. No inventes la respuesta.
8. **Entrada del jugador:** los casos que dependen del teclado pueden diseñarse, pero no cuentan como implementables (ver sección 6.3).

6.3 Qué casos implementar

Implementá **al menos 4** de los 6 casos, entre los que tiene que haber:

- 1 positivo (el de varios efectos, regla 5);
- 1 negativo;
- 1 de borde;
- 1 caso más, de cualquier tipo.

No implementes casos marcados con ⚠ ni casos que dependan del teclado. Si uno de tus casos necesita un objeto que no existe en el juego (un **dummy**, como en el caso INT-AST-10 de clase), podés crearlo en el test. Indicá en la ficha que es un dato de prueba artificial.

6.4 Reglas de calidad para los tests implementados

1. **Nombres:** cada método se llama con el formato `Flujo_Condición_Resultado`.
2. **Componentes reales:** cada test usa el prefab `Game` y al menos 2 componentes reales. No se permite reemplazar por dobles los componentes de la costura que se verifica.
3. **Esperas:**
   - Para esperar un efecto (destrucción, cambio de estado), usá una espera **por condición** con tiempo máximo.
   - Para esperar una colisión, usá `WaitForFixedUpdate`.
   - `WaitForSeconds` solo se admite cuando lo que se mide es el paso del tiempo (por ejemplo, un intervalo), con una justificación en un comentario.
4. **Objetos destruidos:** para comprobar si un objeto de Unity fue destruido, usá `obj == null` dentro de `Assert.IsTrue` o `Assert.IsFalse`.
5. **Mensajes:** cada `Assert` lleva un mensaje que dice qué se esperaba.
6. **Conteos:** si el test cuenta objetos, cuenta **solo los que creó el propio test**, no todos los de la escena.
7. **Independencia:** ningún test depende de que otro se haya ejecutado antes.

7. Análisis de resultados

Tabla que se completa en la Actividad 6, con una fila por test:

Columna	Qué escribir
Test	Archivo y nombre del método
Resultado	Pasa / Falla, con el mensaje de falla textual
¿Por qué?	La causa, siguiendo la cadena desde el assert hasta el código del juego (archivo y línea)
¿Falla del test o del sistema?	Cuál de las dos, y qué revisaste para descartar la otra (preparación del escenario, esperas, estado inicial, respaldo del resultado esperado)
¿El resultado esperado tiene respaldo de diseño?	Sí (indicar cuál), No o ⚠ (pregunta de diseño)
Componente o interacción involucrada	La costura entre componentes donde se origina el comportamiento
Evidencia	La evidencia **propia** que usaste: captura, mensaje de falla, `Debug.Log`, resultado de una corrida de la Actividad 4 o 5
Modificación propuesta	Qué cambiarías en el juego o en el test (etiqueta **PROPUESTO**; no se aplica)
Etiqueta	VERIFICADO / UNKNOWN para la conclusión de la fila

Para los tests que **pasan**, la columna «¿Por qué?» explica por qué se puede confiar en ese resultado. Hay que citar la Actividad 5.

8. Entregables

Se entrega **un único archivo comprimido** `TP-Integracion-<Apellido>.zip` con:

#	Archivo	Contenido
1	`IntegracionTP_<Apellido>.cs`	Todos tus tests, la fixture y el test auxiliar de la semilla. Es el **único** archivo de código que se entrega.
2	`Informe-<Apellido>.pdf`	Informe con las secciones 1 a 7, una por actividad, en ese orden. Extensión máxima: 15 páginas sin contar capturas.
3	`evidencias/`	Las capturas citadas en el informe, con nombres que indiquen la actividad (por ejemplo, `A4-R3-corrida2.png`).

No se entregan el proyecto completo ni otros scripts.

El docente copiará el archivo `.cs` en una copia limpia de `asteroide-final` y ejecutará la suite. Por eso, el archivo tiene que **compilar y ejecutarse sin ningún otro cambio en el proyecto**.

9. Criterios de evaluación

Puntaje total: **100 puntos**. Cada ítem se califica como **cumple** (puntaje completo) o **no cumple** (0). Solo admiten puntaje parcial los ítems cuyo puntaje se indica «por flujo», «por ficha» o «por pregunta».

Condición de aprobación del bloque de implementación (criterios C, D y E): el archivo `.cs` compila en una copia limpia del proyecto. Si no compila, los criterios C, D y E se califican con 0.

A · Análisis de componentes (10 puntos)

Ítem	Puntos
A1. Los 3 flujos tienen diagrama con archivo y línea en cada llamada entre componentes	2 por flujo (6)
A2. Cada flujo tiene al menos un efecto sin verificar, justificado con los asserts existentes	2
A3. Las fuentes de estado compartido están ubicadas en el código (archivo y línea)	2

B · Diseño de casos (20 puntos)

Ítem	Puntos
B1. Las 6 fichas tienen los 8 campos completos	1 por ficha (6)
B2. Se cumple la distribución: ≥2 positivos, ≥2 negativos, ≥2 bordes en dimensiones distintas, los 3 flujos cubiertos	4
B3. Ningún caso repite un test existente y como máximo 1 caso viene del catálogo de clase	3
B4. Cada caso indica el estado inicial y el resultado esperado es distinto de ese estado	4
B5. Hay al menos 1 positivo con 2 o más efectos sobre componentes distintos	3

C · Calidad de los tests (20 puntos)

Ítem	Puntos
C1. Hay al menos 4 tests implementados con la distribución de 6.3, y cada uno corresponde a una ficha	4
C2. Nombres `Flujo_Condición_Resultado` y mensaje en cada `Assert`	3
C3. Cada test usa el prefab `Game` y al menos 2 componentes reales	3
C4. Esperas según la regla 6.4.3 (cada `WaitForSeconds` con su justificación)	4
C5. Comprobación de destrucción con `obj == null` y conteos solo de objetos propios	3
C6. Los tests se comportan según su ficha en la ejecución del docente: pasan los que deben pasar y los que fallan coinciden con lo declarado en la Actividad 6	3

D · Independencia y reproducibilidad (15 puntos)

Ítem	Puntos
D1. La fixture usa `[UnitySetUp]` / `[UnityTearDown]`, limpia lo que el test creó y espera los frames necesarios	4
D2. Cada decisión de la fixture está explicada con el problema y la línea del código que la motiva	3
D3. Están los resultados completos de R1, R2 y las 3 repeticiones de R3	4
D4. Están los logs de estado compartido y el experimento de la semilla, con conclusiones etiquetadas	4

E · Comprobación de que los tests pueden fallar (10 puntos)

Ítem	Puntos
E1. Cada test propio tiene un cambio temporal documentado y una captura en rojo	6
E2. Hay una predicción registrada **antes** de cada ejecución y su comparación con el resultado real	2
E3. Se identifica si algún cambio no fue detectado por los tests existentes y se explica qué significa	2

F · Análisis de resultados e interpretación de fallos (15 puntos)

Ítem	Puntos
F1. Están analizados todos los tests que fallan en la última R3, más 2 tests propios que pasan	4
F2. Cada falla está clasificada como del test o del sistema, con lo que se revisó para descartar la otra opción	4
F3. Cada fila cita evidencia propia y la cadena causal llega hasta archivo y línea	4
F4. Cada fila tiene una modificación propuesta, etiquetada como PROPUESTO	3

G · Documentación y conclusiones (10 puntos)

Ítem	Puntos
G1. Todas las afirmaciones sobre el comportamiento tienen etiqueta de estado, y las VERIFICADO tienen evidencia	4
G2. Las respuestas de la sección 11 citan resultados concretos de las actividades	1 por pregunta (5)
G3. La entrega respeta la estructura de la sección 8	1

Penalizaciones

Situación	Efecto
Se presenta como VERIFICADO algo que figura como UNKNOWN sin aportar evidencia nueva	−2 por cada caso
El archivo entregado depende de cambios en el juego o en los tests existentes	Los criterios C, D y E se califican con 0

10. Restricciones

No se puede modificar

- Los scripts del juego: `Assets/Scripts/*.cs`.
- Los prefabs (`Assets/Resources/Prefabs/Game.prefab` y `Assets/Prefabs/*.prefab`) y las escenas.
- Los tests existentes: `Assets/Tests/TestSuite.cs` y `Assets/Tests/IntegrationHypothesesTests.cs`.
- Los assembly definitions (`GameAssembly.asmdef` y `Tests.asmdef`), `Packages/manifest.json` y `ProjectSettings/`.
- No se instalan paquetes adicionales.

Excepción

En la **Actividad 5**, y **solo** ahí, se permiten cambios **temporales** en `Assets/Scripts/*.cs` para reproducir un defecto. Cada cambio se documenta y se revierte antes de seguir. **Ninguna** corrección del juego forma parte de la entrega: las correcciones se proponen por escrito (etiqueta PROPUESTO) en la Actividad 6.

Qué se puede crear

Únicamente el archivo `Assets/Tests/IntegracionTP_<Apellido>.cs`. Dentro de ese archivo se pueden crear objetos auxiliares de prueba, como un dummy con collider.

Fuera del alcance de este TP

No se exige el uso de estas herramientas y no suman puntos:

- Input System;
- tests parametrizados;
- `LogAssert`;
- el atributo `[Order]`;
- ejecución en batchmode o en CI;
- reportes de cobertura.

11. Preguntas finales

Respondé cada pregunta citando al menos un resultado concreto de tus actividades.

1. Elegí uno de tus tests implementados: ¿qué costura verifica y qué defecto de integración detectaría que una prueba unitaria de uno solo de sus componentes no detectaría? Respaldalo con lo que observaste en la Actividad 5.
2. Si tus tests pasaron en todas las corridas de la Actividad 4, ¿eso demuestra que son independientes? ¿Qué demostraste exactamente y qué quedó como UNKNOWN?
3. Elegí un cambio temporal de la Actividad 5 que ningún test existente haya detectado (si no hubo ninguno, el que detectaron menos tests). ¿Qué dice ese resultado sobre una suite que estaba en verde?
4. Elegí una falla de la Actividad 6: ¿cómo determinaste si era del test o del sistema? ¿Qué información necesitarías del diseño del juego para confirmarlo?
5. Elegí una espera que reemplazaste o que justificaste en tus tests: ¿qué riesgo concreto evita la forma de espera que elegiste frente a una espera fija, en este proyecto?